

Source Code to UML Class Diagram

Learning Objectives

By the end of this lecture, students will be able to:

1. Understand the purpose and benefits of reverse engineering UML diagrams from source code
2. Identify key UML elements in different programming languages
3. Extract class relationships from code manually
4. Use tools to automate UML generation
5. Apply this knowledge to documentation and system understanding

Why This Matters

text

"As a software engineer, you'll spend 70% of your time READING code, not writing it.

UML diagrams from source code are your map through someone else's (or your own!) code maze."

PART 1: FOUNDATIONS

What is Reverse Engineering in UML?

plaintext

Forward Engineering: Design → Code
Reverse Engineering: Code → Design (We are HERE!)

Round-trip Engineering: Both directions

Why Extract UML from Code?

java

```
// Consider this code WITHOUT a diagram:
public class A {
    private List<B> items;
    public void process(B b) { /* ... */ }
}
```

```
// Questions you might ask:
// - What does A do with B?
// - Is this composition or aggregation?
```

```
// - Are there other classes involved?
```

Real-world scenarios:

- Scenario 1: You join a team with 500,000 lines of legacy code
- Scenario 2: You need to document a system you built 2 years ago
- Scenario 3: Your manager asks: "How does Module X connect to Module Y?"

Quick UML Refresher

text

Class Notation:

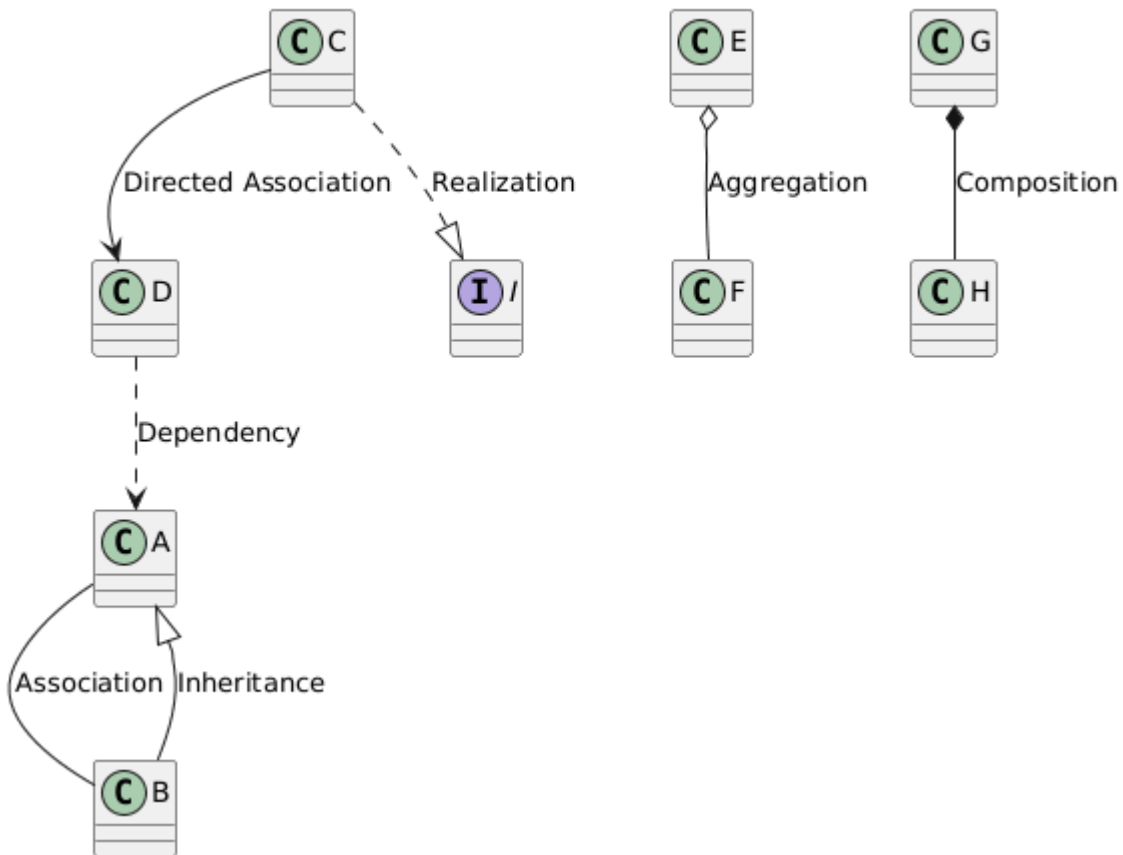
```
+-----+
|      ClassName      |
+-----+
| - privateAttribute: Type |
| + publicAttribute: Type  |
| # protectedAttribute: Type |
+-----+
| + publicMethod(): ReturnType |
| - privateMethod(): void    |
+-----+
```

Relationships:

#	Relationship	UML Symbol	Line Style	Meaning	Example
---	--------------	------------	------------	---------	---------

1	Association	A — B	Solid line	General structural connection between two classes	Student — Course
2	Directed Association	A —> B	Solid line with arrow	One class knows/uses another in one direction	Teacher —> Course
3	Aggregation	Whole o— Part	Solid line + Hollow diamond	Whole-part relationship (part can exist alone)	Department o— Teacher
4	Composition	Whole *— Part	Solid line + Filled diamond	Whole-part where part cannot exist alone	Order *— Order Item
5	Generalization (Inheritance)	Child —	> Parent`	Independent	"Is-a" relationship
6	Realization (Interface)	Class —	> Interface`	Independent	Implements interface
7	Dependency	A - - > B	Dashed arrow	One class temporarily uses another	Controller - - > Service

UML Class Diagram - All Relationship Types Demo



PART 2: MANUAL EXTRACTION FROM CODE

Step-by-Step Process

Step 1: Identify Classes

Look for:

- Java: `class` keyword
- C++: `class` or `struct`
- Python: `class` keyword
- C#: `class` keyword

java

// Code

```
public class Customer {  
    private String name;
```

```

        private String email;

        public void placeOrder(Order order) {
            }
    }

    public class Order {
        private int id;
        private List<Item> items;
        private Customer customer;

        public double calculateTotal() {
            }
    }

    public class Item {
        private String product;
        private double price;
        private int quantity;
    }

```

Extracted Classes:

text		
+-----+	+-----+	+-----+
Customer	Order	Item
+-----+	+-----+	+-----+
- name: String	- id: int	- product: String
- email: String	- items: List<Item	- price: double
+-----+	- customer: Customer	- quantity: int
+ placeOrder()	+-----+	+-----+
+-----+	+ calculateTotal()	+ getters/setters
	+-----+	+-----+

Step 2: Identify Attributes and Methods

Rules of thumb:

- Fields → Attributes
- Methods → Operations
- Static → Underlined
- Abstract → Italic

```

java
public abstract class Vehicle {
    private String licensePlate;           // - licensePlate: String
    protected double speed;                // # speed: double
    public static int vehicleCount;        // + vehicleCount: int
    (underlined)

    public void accelerate() { }           // + accelerate(): void
    private void checkEngine() { }         // - checkEngine(): void
    public abstract void start();          // + start(): void (italic)
}

```

Becomes:

```

text
+-----+
|           Vehicle           |
+-----+
| - licensePlate: String      |
| # speed: double             |
| + vehicleCount: int         |
+-----+
| + accelerate(): void        |
| - checkEngine(): void       |
| + start(): void             |
+-----+

```

Step 3: Identify Relationships

Association (Uses)

```

java
// Look for method parameters or return types
public class Driver {
    public void drive(Car car) { // Association: Driver uses Car
        car.start();
    }
}

```

```

text
[Driver]----->[Car]

```

Aggregation (Has-a, Optional)

java

```
// Look for collections or references that can exist independently
public class Team {
    private List<Player> players; // Players can exist without Team

    public Team() {
        players = new ArrayList<>();
    }
}
```

}

text

[Team] ◇-----> [Player]

Composition (Owns, Exclusive)

java

```
// Look for objects created and destroyed together
public class House {
    private Room livingRoom; // Room cannot exist without House

    public House() {
        livingRoom = new Room(); // Created together
    }
}
```

}

text

[House] ●-----> [Room]

Inheritance (Is-a)

java

```
// Look for 'extends' keyword
public class SavingsAccount extends Account {
    // inherits from Account
}
```

}

text

[SavingsAccount] ----> [Account]

Interface Implementation

```
java
// Look for 'implements' keyword
public class ArrayList<T> implements List<T> {
    // implements List methods
}

text

[ArrayList]---->[List<<interface>>]
```

Classroom Exercise

Given this code, draw the UML class diagram:

```
java

public interface Drawable {
    void draw();
}

public abstract class Shape {
    protected String color;
    public abstract double getArea();
}

public class Circle extends Shape implements Drawable {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double getArea() {
```

```
        return Math.PI * radius * radius;
    }

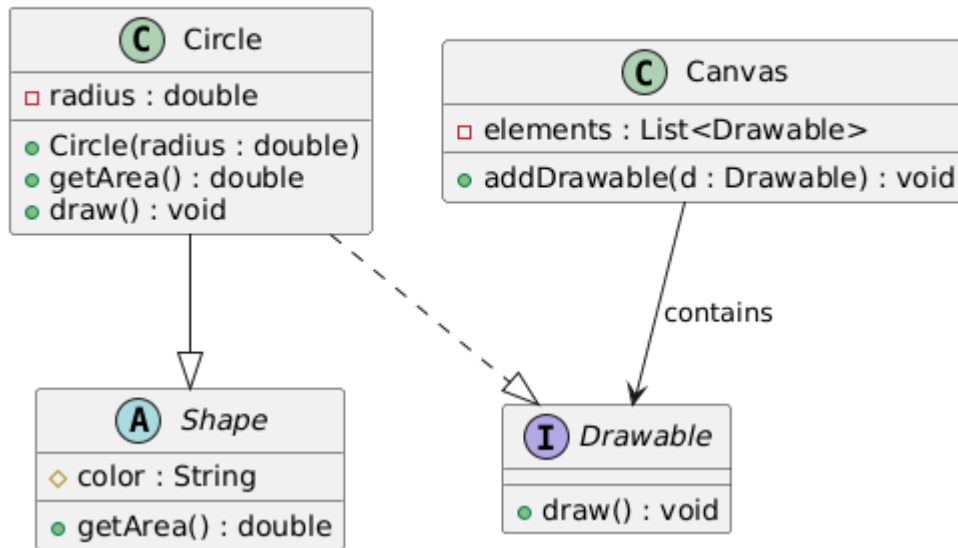
    @Override
    public void draw() {
        System.out.println("Drawing circle");
    }
}

public class Canvas {
    private List<Drawable> elements;

    public void addDrawable(Drawable d) {
        elements.add(d);
    }
}
```

Solution:

Shape Drawing System - Class Diagram



PART 3: AUTOMATED TOOLS

Popular Tools Comparison

Tool	Language	Pros	Cons	Price
IntelliJ Diagrams	Java/Kotlin	Built-in, live updates	Limited export	Free
Visual Paradigm	Multiple	Comprehensive	Heavy, expensive	paid
PlantUML	Any	Text-based, version control	Manual setup	Free
Doxygen	C++/Java	Documentation + diagrams	Old interface	Free

StarUML	Multiple	Good UI	Limited free version	paid
---------	----------	---------	----------------------	------

Tool Demo 1: IntelliJ IDEA (Quickest)

Steps:

1. Right-click package → Diagrams → Show Diagram
2. Choose Java Class Diagram
3. Use shortcuts:
 - Space - Add class to diagram
 - Delete - Remove from diagram
 - Drag to arrange

Result:

text

[IntelliJ automatically shows:]

- Classes with fields/methods
- Inheritance arrows
- Association arrows
- Package structure

Tool Demo 2: PlantUML (Best for Documentation)

Write this:

```
plantuml
```

```
@startuml
```

```
class Customer {  
    - String name  
    - String email  
    + placeOrder(Order)  
}
```

```
class Order {  
    - int id  
    - List<Item> items  
    + calculateTotal()  
}
```

```
}

class Item {
    - String product
    - double price
}

Customer --> Order : places
Order *--> Item : contains

@enduml
```

Generate:

```
text

[PlantUML generates a professional UML diagram from this text]
```

Tool Demo 3: Python with Pyreverse

```
bash

# Install
pip install pylint

# Generate UML from Python code

pyreverse -o png -p MyProject src/
```

For Python code:

```
python

class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
```

```

        return "Meow!"

class Kennel:
    def __init__(self):
        self.dogs = []

    def add_dog(self, dog):

        self.dogs.append(dog)

```

Pyreverse output:

text

[Automatically generated diagram showing inheritance and aggregation]

Best Practices

DO:

- ✓ Start with high-level overview
- ✓ Hide implementation details initially
- ✓ Focus on public interfaces
- ✓ Use tools for initial draft
- ✓ Manually refine the diagram

DON'T:

- ✗ Show every private method
 - ✗ Include trivial getters/setters
 - ✗ Let tools create unreadable diagrams
- ✗ Forget to update diagrams when code changes

PART 5: REAL-WORLD APPLICATION

Reducing complexity: Understanding a Legacy System

Situation: You inherit a banking system with 10,000+ classes

Process:

1. Run automated tool → Get monster diagram (useless!)
2. Filter by package → Get 50 classes per module
3. Identify key patterns:

```
[Account] <-- [SavingsAccount]
[Account] <-- [CheckingAccount]
[Bank] ◇----> [Account]
[Customer] --> [Bank]
```

4. Now you understand the architecture!

When to Use Which Approach

Situation	Approach
Quick understanding of a single file	Manual + IntelliJ
Documenting a small feature	PlantUML
Understanding package structure	Pyreverse/Doxygen
Complete system documentation	Visual Paradigm
Code review preparation	Manual + highlighting

SUMMARY & KEY TAKEAWAYS

The 5-Step Process

1. Scan for classes, interfaces, enums
2. Extract attributes and methods
3. Identify relationships:
 - extends → Inheritance
 - implements → Realization
 - has-a field → Aggregation/Composition

- method parameter → Association

4. Use tools for automation

5. Refine for readability

Remember:

- Tools are helpers, not replacements for understanding
- Start broad, then zoom into details
- Hide complexity - show what matters
- Keep diagrams updated with code changes
- Different stakeholders need different views

Final Thought

"A UML diagram from code is like a map of a city you're about to explore.

It won't show you every building, but it'll keep you from getting lost."